



Memoria

Laboratorio Prácticas

Algoritmia

Alberto Cienfuegos Álvarez

UO308387



Tabla de Contenidos

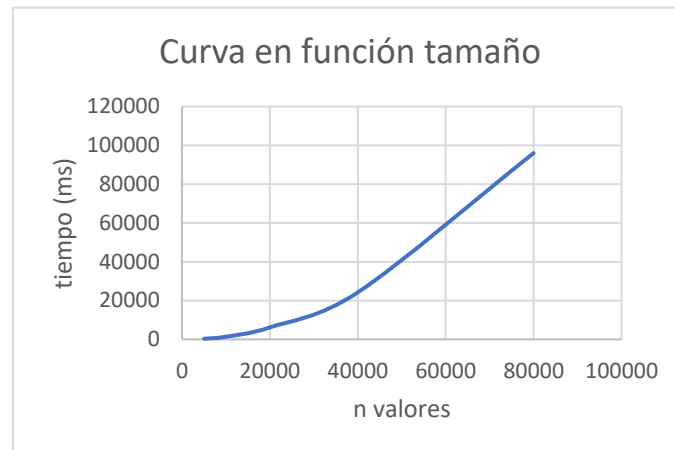
Práctica 0: Introduccion	3
Práctica 1.1: Toma tiempos algoritmos	6
Práctica 1.2: Comparacion tiempos algoritmos	11
Práctica 2: Algotirmos de ordenación	15
Práctica 3: Divide y venceras	19
Práctica 4: Algotirmo voraz, Coloreado de mapas	24
Práctica 5: Programación dinámica, Ferry.....	26
Práctica 6: Backtraking, Almacenaje de contenedores	27
Práctica 6 Extra: Backtraking, Problema de NReinas	28
Práctica 7: Ramifica y poda	29



----- PRACTICA 0 -----

FACTOR1: TAMAÑO DEL PROBLEMA

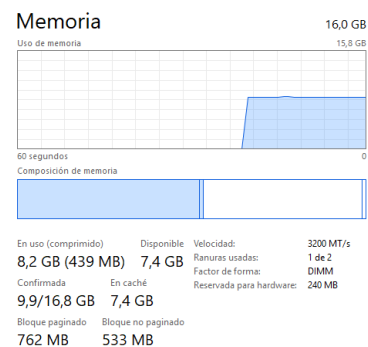
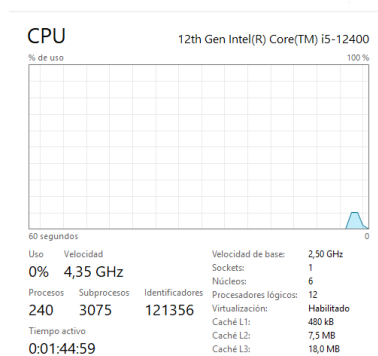
PROG / TIEMPO	5000	10000	20000	40000	80000
PythonA1.py	337	1387	6210	24234	96061



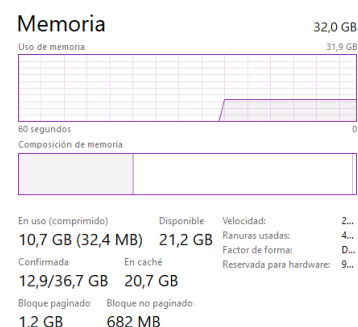
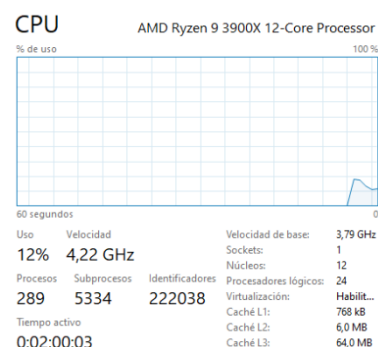
FACTOR2: POTENCIA DEL ORDENADOR

PC / TIEMPO	5000	10000	20000	40000	80000
PC 1	337	1387	6210	24234	96061
PC 2	574	2329	9460	37711	152489

PC 1



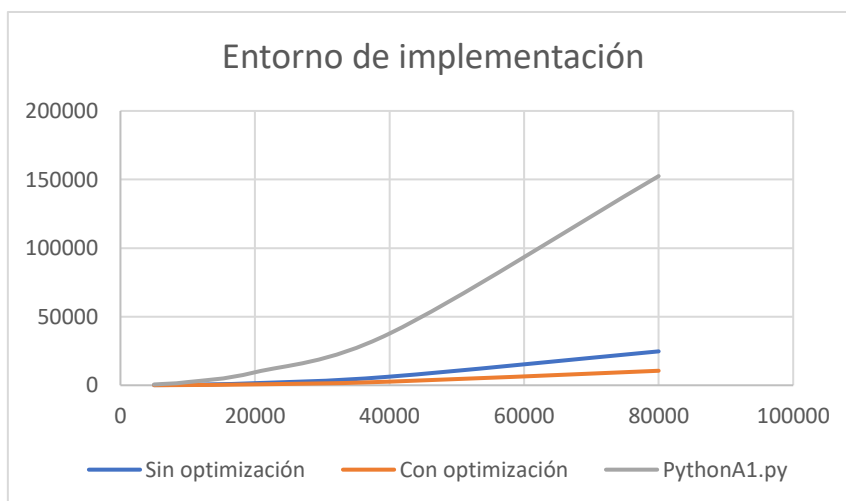
PC 2





FACTOR3: ENTORNO DE IMPLEMENTACIÓN

JavaA1.java / TIEMPO	5000	10000	20000	40000	80000
Sin optimización	106	391	1533	6273	24703
Con optimización	53	165	659	2640	10570
PythonA1.py	574	2329	9460	37711	152489



Se puede observar cómo ambas ejecuciones en Java son mucho más rápidas que el programa Python, así mismo ejecutando el programa con el JIT de java (optimizado) es prácticamente el doble de rápido que sin optimizar.

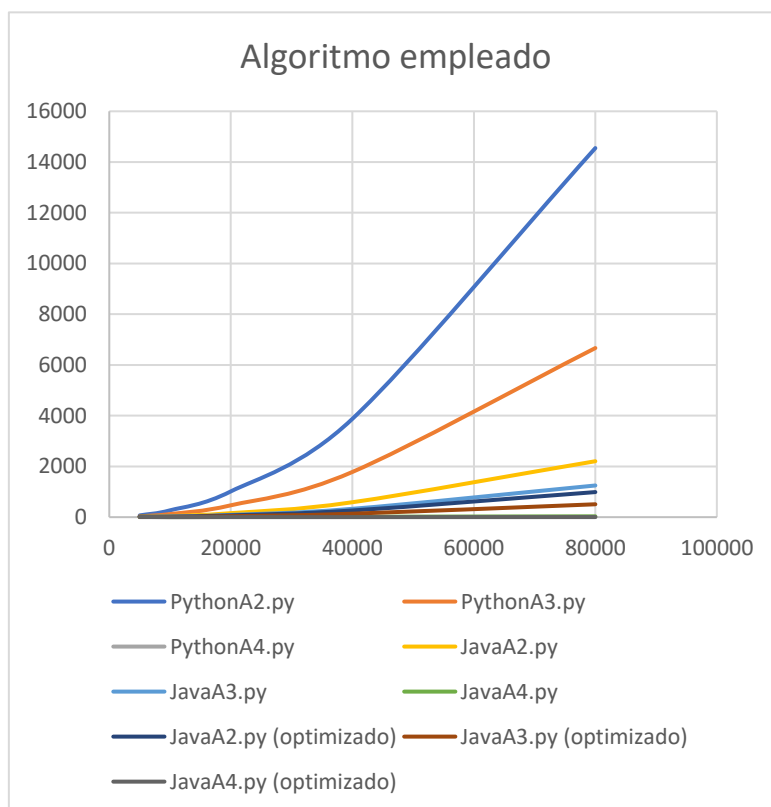
FACTOR4: ALGORITMO EMPLEADO

Python

PROG / TIEMPO	5000	10000	20000	40000	80000
PythonA2.py	70	269	1015	3874	14550
PythonA3.py	33	123	467	1781	6667
PythonA4.py	1	1	3	6	11

Java

PROG / TIEMPO	5000	10000	20000	40000	80000
JavaA2.py	22	44	158	588	2205
JavaA3.py	17	25	89	329	1248
JavaA4.py	12	4	8	17	33
JavaA2.py (optimizado)	17	20	71	263	989
JavaA3.py (optimizado)	15	10	38	137	508
JavaA4.py (optimizado)	12	2	4	2	3



De los datos obtenidos podemos observar que el algoritmo que se emplee marcara mucho la diferencia del tiempo de ejecución, siendo los algoritmos A4 los mejores, incluso el de java se puede mejorar su tiempo si le incorporamos la optimización. El mejor de todos a sido el PythonA4.

----- PRACTICA 1.1 -----

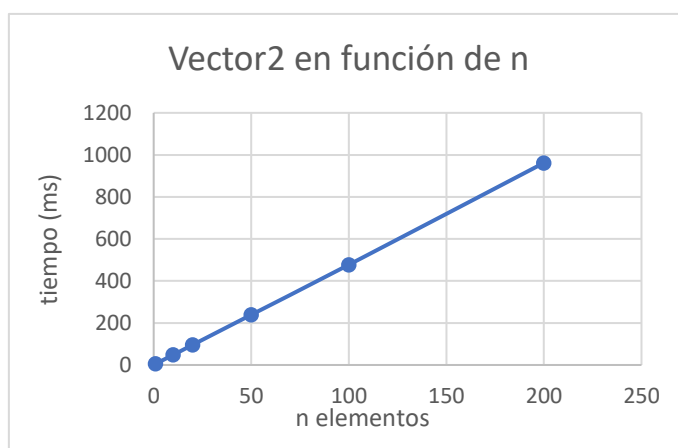
FASE 2: TOMA DE TIEMPOS DE EJECUCIÓN

¿Calcular cuántos años más podremos seguir utilizando `currentTimeMillis()` como forma de contar?

Devuelve un valor de tipo long (64 bits) por tanto tendríamos un total de 2^{64} valores posibles, lo cual pasando a años sería: $(2^{64}) / (1000 * 60 * 60 * 24 * 365)$ aproximadamente **5,58*10⁸ años**. Para hacer el cálculo dividimos los valores entre: 1000 (pasar a segundos), 60 (pasar a minutos), 60 (pasar a horas), 24 (pasar a días), y 365 (pasar a años).

PROG / n Valor	1M	10M	20M	50M	100M	200 M
Vector2	6	48	96	239	477	962

Ejecución del programa Vector2 para varios valores de n:



¿Por qué a veces el tiempo medido sale 0?

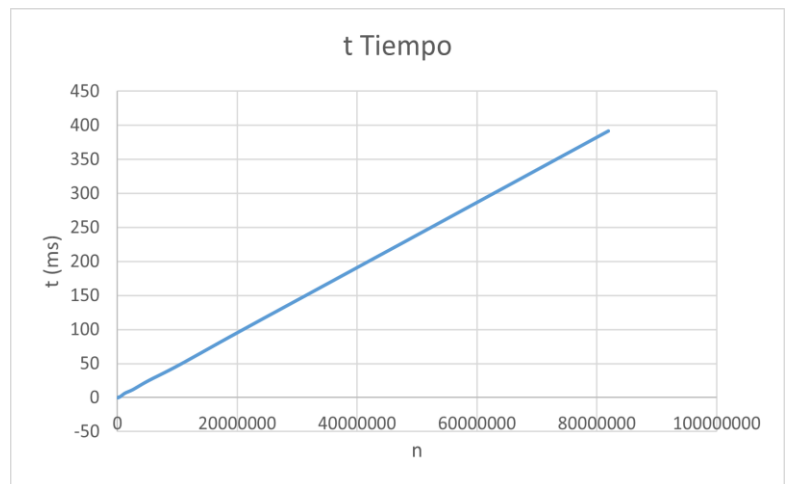
El tiempo medido puede salir cero cuando se utilizan tamaños pequeños de n para el vector, debido a que la función `currentTimeMillis()` tiene una precisión de milisegundos, por tanto si la operación del algoritmo tarda menos de 1 ms el tiempo de inicio y de fin de la llamada a `currentTimeMillis()` será el mismo y por tanto su diferencia será cero.

¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables?

A partir de valores superiores al millón empezamos a tener tiempo que podemos considerar fiables, ya que probando valores de n por debajo del millón como puede ser 500k o 250k no obtenemos linealidad en las mediciones.

FASE 3: CRECIMIENTO DEL TAMAÑO DEL PROBLEMA

n	tTiempo
10000	0
20000	0
40000	0
80000	0
160000	1
320000	1
640000	3
1280000	7
2560000	12
5120000	25
10240000	49
20480000	98
40960000	196
81920000	392



La complejidad total de Vector3 es lineal $O(n)$ porque a pesar de que los saltos dentro del bucle generan una complejidad logarítmica, las operaciones que hay dentro del bucle como la suma son lineales, por tanto también la total será lineal.

FASE 4: TOMA DE TIEMPOS PEQUEÑOS (<50ms)

n / Repeticiones	1	10	100	1000	100000	10000000
1k	0	1	6	63	5934	614887
2k	0	1	12	120	12001	1231352
4k	0	2	24	237	23977	2463439
8k	1	4	49	477	47053	4941750
16k	1	10	97	962	94537	9877132
32k	2	19	193	1918	188542	FdT
64k	4	38	388	3876	377918	FdT
128k	7	78	791	7748	753636	FdT
256k	15	154	1584	15303	1513864	FdT
512k	30	306	3100	30608	3037986	FdT
1024k	60	620	6407	61152	6075718	FdT
2048k	123	1230	12546	123526	12180503	FdT
4096k	244	2540	24954	248667	24951432	FdT
8192k	508	5093	50814	497672	49329834	FdT

*por cuestiones obvias los resultados en rojo han sido “fakeados” debido a que no puedo permitir que mi ordenador se encuentre más de 12 horas encendido. Además de la última fila con 10M tampoco daba tiempo a que se terminase tendiendo el ordenador encendido durante todo el día (FdT).



¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?

Para un tamaño de problema que se vaya multiplicando por 2, es el caso original de Vector4 cuyos valores están reflejados en la tabla anterior. Podemos ver como si cogemos un número de repeticiones que haga los tiempos estables, como el de 100 repeticiones, podemos concluir que el tiempo cada vez que se duplica el problema se va aproximadamente duplicando también.

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para $k=3$ y $k=4$ y compruebe los tiempos obtenidos.)

Para 100 repeticiones multiplicando el problema por k:

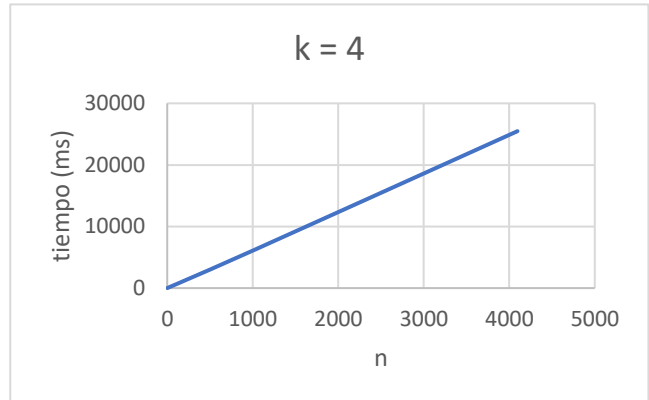
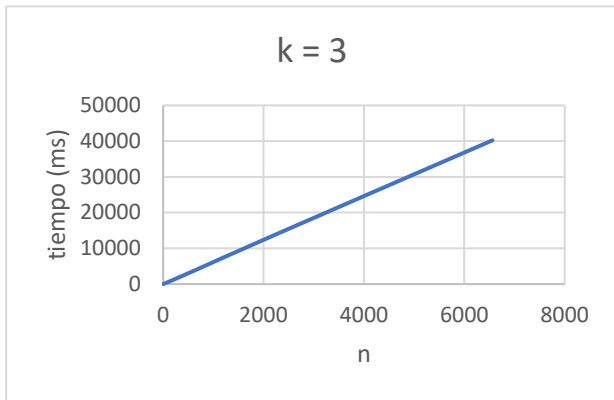
n / k	3
1k	6
3k	18
9k	55
27k	165
81k	482
243k	1453
729k	4448
2187k	13540
6561k	40216

n / k	4
1k	6
4k	27
16k	98
64k	391
256k	1537
1024k	6235
4096k	25480

Utilizando otro k para ir multiplicando el tamaño del problema podemos observar también como hay una linealidad en la forma en la que el tiempo se va multiplicando, concluyendo así como el tiempo se ve afectado por el factor k, es decir cada vez que se multiplica el tamaño del problema por k también el tiempo se verá multiplicado aproximadamente por k

¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal $O(n)$?

Los tiempo obtenidos son los que se esperaban ya que ambos algoritmos modificando su tamaño de problema mantienen la linealidad esperada.

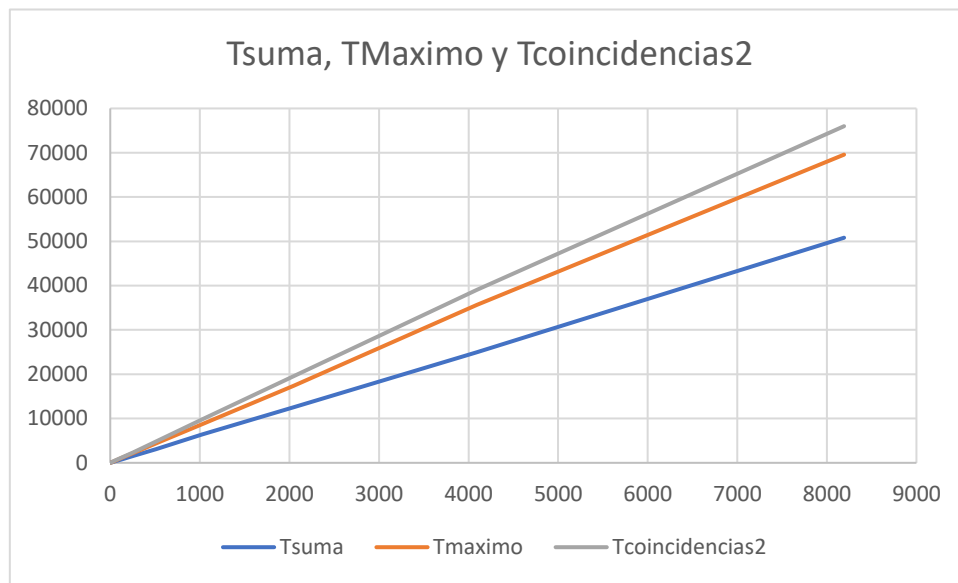


FASE: TOMA DE TIEMPOS DE ALGORITMOS

En estas mediciones se han utilizado un total de 100 repeticiones por programa, a excepción del Tcoincidencias1, que he utilizado 1 repetición porque si no el tiempo sería muy grande.

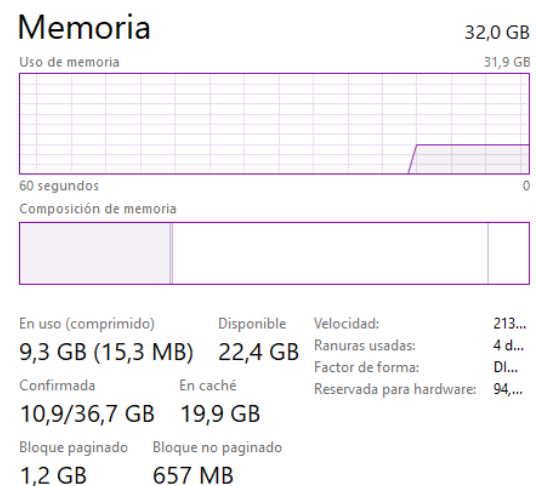
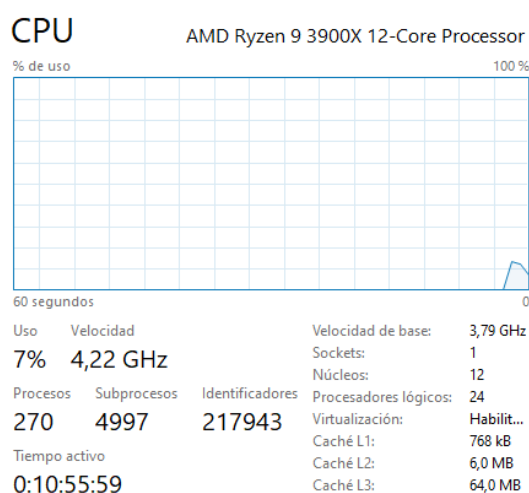
n	Tsuma	Tmaximo
1k	6	9
2k	12	17
4k	24	33
8k	49	67
16k	97	133
32k	193	270
64k	388	548
128k	791	1100
256k	1584	2187
512k	3100	4348
1024k	6407	8745
2048k	12546	17355
4096k	24954	35699
8192k	50814	69573

n	Tcoincidencias1	Tcoincidencias2
1k	740	9
2k	2949	19
4k	11963	38
8k	47709	75
16k	190399	147
32k	FdT	294
64k	FdT	588
128k	FdT	1174
256k	FdT	2368
512k	FdT	4811
1024k	FdT	9796
2048k	FdT	19548
4096k	FdT	39093
8192k	FdT	75998



Observado los datos de la tabla y la gráfica podemos ver que los tiempos obtenidos sintonizan con las complejidades temporales que habíamos deducido para las operaciones, siendo las nombras en la gráfica $O(n)$, es decir lineales. Para el caso de Tcoincidencias1, observando los datos podemos concluir que la tendencia que tienen es exponencial porque a medida que aumenta la carga el tiempo se dispara, siendo así $O(n^2)$ como era de esperar.

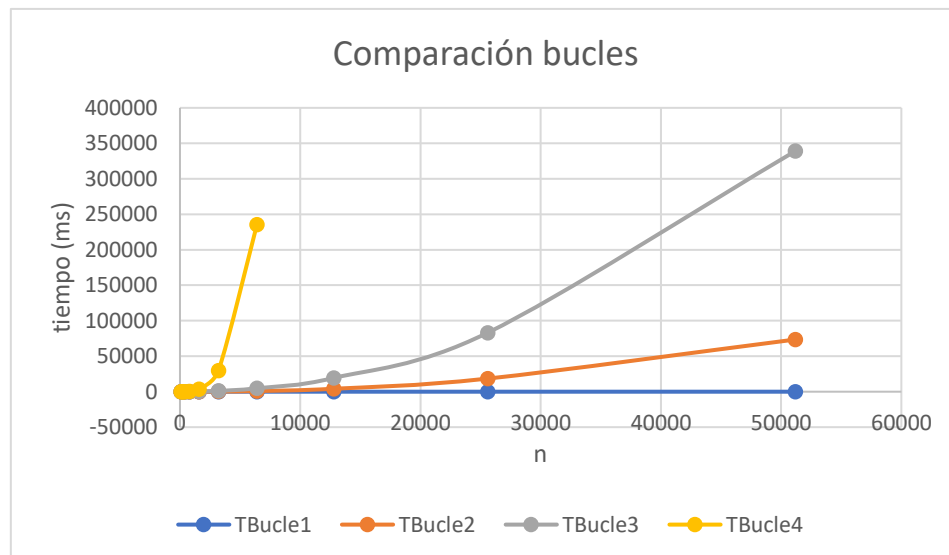
ESPECIFICACIÓN ORDENADOR:



----- PRACTICA 1.2 -----

FASE 1: TOMA DE TIEMPOS DE EJECUCIÓN

n / Tiempo	TBucle1	TBucle2	TBucle3	TBucle4
100	0	0	1	1
200	0	1	3	8
400	0	3	15	60
800	0	14	63	460
1600	0	50	263	3743
3200	1	229	1143	29828
6400	1	925	4839	235406
12800	1	4163	19438	1935519
25600	4	18499	83171	FdT
51200	8	73556	339259	FdT



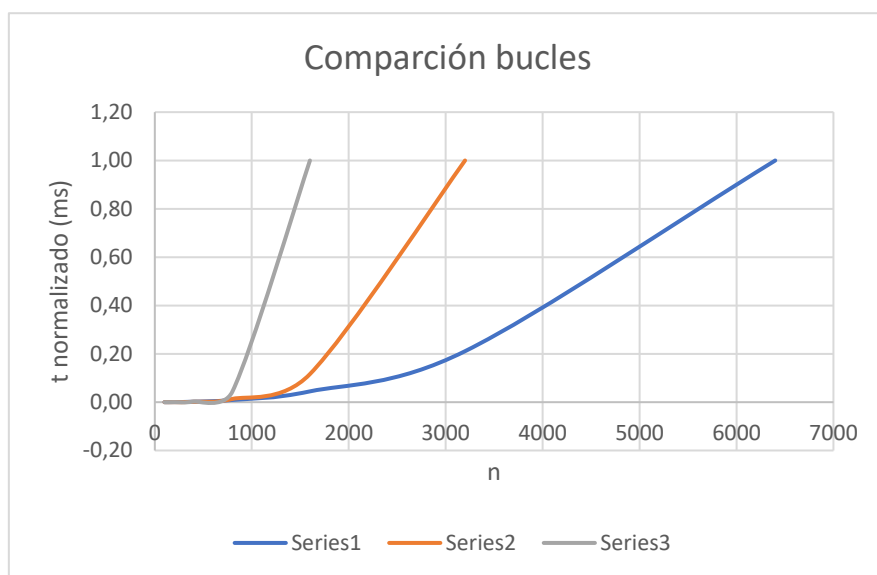
Complejidad teórica $O()$: B1 -> $n \cdot \log(n)$, B2 -> $n^2 \cdot \log(n)$, B3 -> $n^2 \cdot \log(n)$, B4 -> n^3

Los tiempos de todos los algoritmos de bucles concuerdan con las complejidades esperadas, lo único a destacar es la explicación de porque B2 y B3 a pesar de tener misma complejidad teórica luego en la práctica los tiempos son muy diferentes (esto se explica más adelante).

FASE 2: CREACIÓN DE BUCLES DE COMPLEJIDAD DADA

Bucle5 -> $O(n^2 \cdot (\log(n))^2)$ **Bucle6** -> $O(n^3 \cdot \log n)$ **Bucle7** -> $O(n^4)$

n / Tiempo	TBucle5	TBucle6	TBucle7
100	1	8	21
200	4	65	284
400	22	571	4249
800	106	5034	65509
1600	514	43452	1497482
3200	2427	373887	FdT
6400	11510	FdT	FdT



Los tiempos obtenidos concuerdan con lo esperado, ya que en la gráfica podemos observar claramente el factor logarítmico que tienen bucle 5 y 6, y luego donde realmente vemos la diferencia entre cada algoritmo es en la tendencia de cada bucle marcado por el número que eleva la x, siendo el peor caso el bucle 7 al tener x^4 .

FASE 3: RELACIÓN TEMPORAL DE ALGORITMOS

Caso1: Dos algoritmos de diferente complejidad

n / Tiempo	TBucle1(t1)	TBucle2(t2)	t1/t2
100	0	0	inf
200	0	1	0
400	0	3	0
800	0	14	0
1600	0	50	0
3200	1	229	0,0043668
6400	1	925	0,0010811
12800	1	4163	0,0002402
25600	4	18499	0,0002162
51200	8	73556	0,0001088

Los tiempos de t_1 y t_2 son los esperados ya que ambas complejidades tienen un producto entre n con un logaritmo de n , con la diferencia que en t_2 la n es cubica, por tanto marca la gran diferencia en el aumento del tiempo. A medida que aumentamos el tamaño del problema (n), el tiempo que tarda t_2 va aumentando a una mayor proporción que t_1 , es por eso que el cociente entre t_1 y t_2 tiende a 0, por tanto es correcto.

Caso2: Dos algoritmos de misma complejidad

n / Tiempo	TBucle3(t_3)	TBucle2(t_2)	t_3/t_2
100	1	0	0
200	3	1	3
400	15	3	5
800	63	14	4,5
1600	263	50	5,26
3200	1143	229	4,99
6400	4839	925	5,23
12800	19438	4163	4,66
25600	83171	18499	4,49
51200	339259	73556	4,61

Los tiempos son los esperados ya que ambos bucles tienen la misma complejidad teórica, pero la razón por la que bucle 3 tarda aproximadamente entre 4 y 5 veces más que bucle 2, es debido a que luego en la práctica si aplicamos conversiones matemáticas del algoritmo en sumatorios, fuera de las variables n encontramos unas constantes multiplicativas diferentes en función de cómo este construido el algoritmo. Entonces podemos calcular que la constante de bucle 3 es mucho mayor que la de bucle 2 a pesar de tener la misma complejidad teórica y es por eso que del cociente entre ambas obtenemos una constante que al ser mayor que 1 podemos afirmar que el algoritmo del numerador es decir t_3 , es mucho más lento que t_2 .

Caso2: Mismo algoritmo en entornos de desarrollo diferentes

n / T Bucle4	Python	Java sin optimización	tPython/tJava
200	37	8	4,625
400	279	60	4,650
800	2403	460	5,224
1600	21075	3743	5,630
3200	188365	29828	6,31
6400	FdT	235406	> 6,31



Los tiempos concuerdan con lo esperado, ya que ambos algoritmos en Python y en java podemos observar que tienen un mismo orden de crecimiento, el cual es el esperado a su complejidad de $O(n^3)$. Por otro lado podemos ver que la constante del cociente entre Python y java es mayor que 1, lo que significa que Python es más lento que java, esto se debe principalmente a que Python es un lenguaje interpretado y de tipado dinámico, frente a java que tiene una compilación a bytecode y de tipado estático, entre otras muchas cosas.

----- PRACTICA 2 -----

ALGORITMO 1: BURBUJA

Para todas las tablas se han usado un repetición:

n / Tiempo	t ordenado	t inverso	t aleatorio
10000	425	1887	855
20000	1786	4321	3505
40000	6755	17437	13997
80000	27829	69920	55842
160000	107285	279288	222509

Los tiempos concuerdan con los esperados, en los tres casos podemos ver la complejidad cuadrática debido a que siempre se ejecutan al completo los dos bucles anidados. El algoritmo de burbuja tiene una condición de ir comprobando si el elemento de la izquierda es mayor que el de la derecha entonces si se cumple los intercambia, es por eso que el caso que menos tarda es cuando el vector esta ordenado tal como reflejan los tiempos ya que nunca ejecuta el bloque de dentro del if. El peor caso es cuando el vector esta ordenado inversamente ya que en cada iteración tiene que ejecutar el bloque de dentro del if y el caso medio es cuando es aleatorio que aproximadamente tendrá que ejecutarlo la mitad de las veces.

ALGORITMO 2: SELECCIÓN

n / Tiempo	t ordenado	t inverso	t aleatorio
10000	401	345	387
20000	1595	1380	1585
40000	6308	5503	6254
80000	25299	21945	24950
160000	100790	87503	100450

Los tiempos concuerdan con los esperados, ya que el algoritmo de selección en cada iteración del bucle va a intercambiar en la posición que toque por el menor elemento, es por eso que en los tres casos el tiempo es aproximadamente el mismo, no habiendo una caso mejor que otro. Además podemos observar cómo se reflejan en los tiempos la complejidad cuadrática debido a tener dos bucles anidados.

ALGORITMO 3: INSERCIÓN

n / Tiempo	t ordenado	t inverso	t aleatorio
10000	0	682	349
20000	0	2742	1363
40000	0	10962	5402
80000	2	43631	21710
160000	4	173218	86436

Los tiempos concuerdan con los esperados, ya que el algoritmo tiene implementado dentro del primer bucle un bucle while, que realiza la operación de intercambio mientras se cumpla una determinada condición, es por eso que el mejor caso es cuando el vector ya está ordenado, presentando una complejidad lineal y por otro lado estarían los vectores inversos y aleatorios que tienen una complejidad cuadrática tardando en general más tiempo a medida que aumentamos en tamaño del problema, siendo el peor caso el inverso ya que debe entrar siempre dentro del bucle while.

ALGORITMO 4: RAPIDO

n / Tiempo	t ordenado	t inverso	t aleatorio
10000	2	3	3
20000	5	6	8
40000	11	12	16
80000	24	26	36
160000	49	53	77
320000	101	111	154
640000	210	233	325
1280000	444	482	698

Los tiempos concuerdan con lo esperado ya que podemos ver cómo se refleja la complejidad cuasi-lineal sobre los tiempos de los tres casos, debido a que el algoritmo realiza particiones del problema de forma recursiva, lo que evita que el algoritmo tenga una complejidad cuadrática.

Proyete, a partir de las complejidades y los datos de las tablas anteriores, ¿cuántos días (1 día= 861400.000 milisegundos) tardaría cada uno de los cuatro métodos (Burbuja, Selección, Inserción y Rápido) en ordenar un vector de $n= 811920.000$ (o sea, $n=80000*2^4=80000*210$), en el caso inicialmente en orden aleatorio?

Para estos tres primeros algoritmos cuadráticos usaremos esta fórmula:

$$t_2 = k^2 * t_1, \text{ siendo } k = n_2 / n_1 = 81.920.000 / 160\,000 = 512$$

Burbuja $O(n^2)$:

$$t_2 = 512^2 * 222509 = 58328383000 \text{ ms} = 677 \text{ días aprox}$$

Selección $O(n^2)$:

$$t_2 = 512^2 * 100450 = 26332365000 \text{ ms} = 305 \text{ días aprox}$$

Inserción $O(n^2)$:

$$t_2 = 512^2 * 86436 = 22658048000 \text{ ms} = 263 \text{ días aprox}$$

Para el caso del algoritmo rápido al ser logarítmico usamos la siguiente formula:

$$t_2 = K * ((\log K + \log n_1) / \log n_1) * t_1, \text{ siendo } k = 512$$

Rápido $O(n \cdot \log n)$:

$$t_2 = 512 * (\log 512 + \log 160000 / \log 160000) * 77 = 59900 \text{ ms} = 1 \text{ minuto aprox}$$

Comparación RAPIDO vs INSERCIÓN para tiempos bajos:

n / Tiempo	t rápido	t inserción	t rápido / t inserción
10	116	66	1,758
15	193	103	1,874
20	250	184	1,359
25	384	259	1,483
30	445	372	1,196
35	549	475	1,156
40	619	528	1,025
45	725	758	0,956
50	850	952	0,893
55	908	1145	0,793
60	1042	1296	0,804
65	1170	1546	0,757
70	1255	1782	0,704
75	1331	2074	0,642
80	1514	2376	0,637
85	1574	2589	0,608
90	1688	2952	0,572
95	1822	3265	0,558
100	1946	3596	0,541

Para n's pequeños es mucho más eficiente el algoritmo de inserción mientras que para n's más grandes es mejor el rápido. Observando el cociente entre ambos podemos decir que a partir del tamaño de problema de $n = 40$ es mucho mejor el algoritmo rápido ya que comienza a tardar mucho menos que el de inserción.

ALGORITMO OPCIONAL: COMBINACIÓN (rápido + inserción)

n / Tiempo	t combinación	t rápido
10	46	116
15	105	193
20	185	250
25	275	384
30	375	445
35	489	549
40	638	619
45	742	725
50	852	850
55	927	908
60	1033	1042
65	1137	1170
70	1270	1255
75	1350	1331
80	1481	1514
85	1618	1574
90	1679	1688
95	1762	1822
100	1920	1946
PROMEDIO	941,26	967,42

Sabiendo a partir de que n es mejor el rápido he implementado un algoritmo que combina ambos según su tamaño de problema y conseguimos mejorar en promedio el tiempo de ejecución de dicho algoritmo.

----- PRACTICA 3 -----

FASE 1: DyV por SUSTRACCIÓN

Para todas las tablas se han usado un repetición:

n / Tiempo	Sustraccion1	Sustraccion2
1000	0	7
2000	0	28
4000	0	103
8000	0	446
En adelante	StackOverflowError	

n / Tiempo	Sustraccion3
20	49
22	164
24	660
26	2685
28	18493

Sustraccion1 coincide con la complejidad temporal esperada ya que los tiempos son todos cero hasta que se termina la pila de ejecución, por tanto refleja la complejidad lineal esperada, ya que tenemos una llamada recursiva y el trabajo fuera de la llamada en de $k = 0$ por tanto con cuerda con $O(n)$.

Sustraccion2 coincide con la complejidad temporal esperada ya que los tiempo tienen un crecimiento cuadrática, reflejando así la complejidad teórica $O(n^2)$, y que $a = 1$ y $k = 1$ por tanto obtenemos el $O(n^2)$.

Sustracción3 coincide con la complejidad temporal esperada ya que los tiempos tienen una crecimiento exponencial, reflejando la complejidad teórica $O(2^n)$, ya que $a = 2$ y $b = 1$.

¿Para qué valor de n dejan de dar tiempo (abortan) las clases Sustracción1 y Sustracción2?

En ambos casos a partir de la interacción donde $n = 16000$, sucede el error StackOverflowError, debido a que hay muchas llamadas recursivas anidadas que saturan y agotan la pila de llamadas.

¿Cuántos años tardaría en finalizar la ejecución Sustracción3 para $n=80$?

Complejidad: $O(a^{(n/b)}) \rightarrow O(2^n)$

Cogemos como muestra el $n = 30$ (42440 ms), entonces el tiempo para $n = 80$ se calcula:

$$42440 * 2^{(80-30)} = 4,7783 * 10^{19} \text{ ms} / 31536000000 = 1,515195 * 10^9 \text{ años}$$

Aproximadamente 1500 millones de años

Tiempos para las clases implementadas con las respectivas complejidades: $O(n^3)$ y $O(3^{(n/2)})$

n / Tiempo	Sustraccion4
100	2
200	19
400	143
800	1116
1600	9013

n / Tiempo	Sustraccion5
30	682
32	2052
34	6253
36	18572
38	56079

¿Cuántos años tardaría en finalizar la ejecución Sustracción5 para n=80?

Complejidad: $O(a^{(n/b)}) \rightarrow O(3^{(n/2)})$

Cogemos como muestra el n = 38 (56079 ms), entonces el tiempo para n = 80 se calcula:

$$56079 * 3^{((80-38)/2)} = 1,5838 * 10^{16} \text{ ms} / 31536000000 = 502231 \text{ años}$$

Aproximadamente medio millón de años

FASE 2: DyV por DIVISIÓN

n / Tiempo	Division1	Division2	Division3
1M	19	343	71
2M	36	696	140
4M	73	1458	288
8M	144	3013	574
16M	329	6496	1157

En división1, tenemos los siguientes parámetros: a = 1, b = 3 y k = 1, por tanto estamos en el caso $a < b^k$ por tanto obtenemos una complejidad teórica de $O(n)$, observando los tiempos vemos como reflejan el crecimiento lineal.

En división2, tenemos los siguientes parámetros: a = 2, b = 2 y k = 1, por tanto estamos en el caso $a = b^k$ por tanto obtenemos una complejidad teórica de $O(n \log n)$, observando los tiempos vemos como reflejan un crecimiento cuasi-lineal.

En división3, tenemos los siguientes parámetros: a = 2, b = 2 y k = 0, por tanto estamos en el caso $a > b^k$ por tanto obtenemos una complejidad teórica de $O(n)$, observando los tiempos vemos como reflejan el crecimiento lineal.

n / Tiempo	Division4	Division5
1000	20	843
2000	77	4236
4000	263	21214
8000	1270	108804

Division4 debe tener una complejidad total de $O(n^2)$ y estar en el caso de $a < b^k$ para conseguirlo necesitamos los siguientes parámetros: $a = 1$, $b = 2$ y $k = 2$.

Division4 debe tener una complejidad total de $O(n^2)$ y estar en el caso de $a > b^k$ para conseguirlo necesitamos los siguientes parámetros: $a = 5$, $b = 2$ y $k = 2$.

En los tiempos de ambos programas podemos ver su crecimiento cuadrático, pero división 5 tarda más que división 4 porque tiene que hacer una mayor cantidad de llamadas recursivas.

FASE 3: Dos ejemplos básicos

Métodos VectorSuma:

Métodos	Tiempo (nano s)
M1	470
M2	1615
M3	2895

Las complejidades de los tres algoritmos son de $O(n)$ pero el método iterativo es más rápido que los otros dos debido a que en este caso en particular al ser un único bucle que va sumando elementos y no realiza ninguna llamada recursiva hace que su tiempo de ejecución sea el menor.

Métodos Fibonacci:

Métodos	Tiempo (nano s)
M1	259
M2	503
M3	645
M4	4213

Las complejidades de los algoritmos son de $O(n)$ es por eso que los tres primeros son prácticamente parecidos en cuanto a tiempos de ejecución, a excepción del cuarto que si

aplicamos cálculos matemáticos podemos encontrar su motivo en que tiene una constante de 1,6 multiplicando a n.

FASE 4: PRÁCTICA DyV

PRIMERA PARTE – Algoritmo trivial

```
C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosTrivial datos32.txt
PUNTOS MÁS CERCANOS: [27,278446, 90,131051] [27,536642, 89,236468]
SU DISTANCIA MÍNIMA = 0,931098

C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosTrivial datos128.txt
PUNTOS MÁS CERCANOS: [91,434401, 16,793360] [91,160139, 17,465655]
SU DISTANCIA MÍNIMA = 0,726086

C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosTrivial datos512.txt
PUNTOS MÁS CERCANOS: [68,474715, 94,638993] [68,511846, 94,653169]
SU DISTANCIA MÍNIMA = 0,039745

C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosTrivial datos2048.txt
PUNTOS MÁS CERCANOS: [27,692774, 55,783120] [27,686908, 55,818825]
SU DISTANCIA MÍNIMA = 0,036184

C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosTrivial datos8192.txt
PUNTOS MÁS CERCANOS: [83,681527, 10,289108] [83,691485, 10,288760]
SU DISTANCIA MÍNIMA = 0,009964
```

n puntos	tiempo (ms)
1024	380
2048	1423
4096	5689
8192	22520
16384	90431

Coincide con la complejidad teorica esperada debido a que el algoritmo de resolucion del problema tiene dos bucles anidados por tanto tiene una complejidad de $O(n^2)$ tal y como reflejan los tiempos.

SEGUNDA PARTE – DyV

```
C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosDyV datos32.txt
PUNTOS MÁS CERCANOS: [27,278446, 90,131051] [27,536642, 89,236468]
SU DISTANCIA MÍNIMA = 0,931098

C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosDyV datos128.txt
PUNTOS MÁS CERCANOS: [91,434401, 16,793360] [91,160139, 17,465655]
SU DISTANCIA MÍNIMA = 0,726086

C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosDyV datos512.txt
PUNTOS MÁS CERCANOS: [68,474715, 94,638993] [68,511846, 94,653169]
SU DISTANCIA MÍNIMA = 0,039745

C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosDyV datos2048.txt
PUNTOS MÁS CERCANOS: [27,686908, 55,818825] [27,692774, 55,783120]
SU DISTANCIA MÍNIMA = 0,036184

C:\Users\Alberto\Desktop\ReposUNI\alg_CienfuegosAlvarezAlbertoUO308387\p3>java -Xint p3.p3p.PuntosDyV datos8192.txt
PUNTOS MÁS CERCANOS: [83,681527, 10,289108] [83,691485, 10,288760]
SU DISTANCIA MÍNIMA = 0,009964
```



n puntos	tiempo (ms)
1024	72
2048	129
4096	261
8192	540
16384	1137

Los tiempos concuerdan con los esperados ya que reflejan un crecimiento cuasi-lineal, lo cual coincide con la complejidad teórica del algoritmo que es $O(n \cdot \log n)$, ya que vamos dividiendo el problema en subproblemas cada vez más pequeños.

----- PRACTICA 4 -----

COLOREADO DE MAPAS

Heurístico aplicado:

En ambas implementaciones (java y python) he utilizado el mismo heurístico siguiendo las instrucciones del algoritmo de clase: recorrer todos los nodos, para cada uno de ellos visitar sus vecinos, si algún vecino ya tiene algún color este pasa a no estar disponible para el nodo actual (por tanto debemos ir guardando los que ya están usados), una vez identificamos los vecinos y sus colores, seleccionamos el primer color que este libre, y así con todo el grafo.

ColoreoGrafo.java

```
n: 4 Tiempo(ms): 2
n: 8 Tiempo(ms): 1
n: 16 Tiempo(ms): 3
n: 32 Tiempo(ms): 3
n: 64 Tiempo(ms): 6
n: 128 Tiempo(ms): 10
n: 256 Tiempo(ms): 17
n: 512 Tiempo(ms): 28
n: 1024 Tiempo(ms): 57
n: 2048 Tiempo(ms): 120
n: 4096 Tiempo(ms): 210
n: 8192 Tiempo(ms): 351
n: 16384 Tiempo(ms): 809
n: 32768 Tiempo(ms): 1861
n: 65536 Tiempo(ms): 4410
```

n / Tiempo	ColoreoGrafo.java
4	2
8	1
16	3
32	3
64	6
128	10
256	17
512	28
1024	57
2048	120
4096	210
8192	351
16384	809
32768	1861
65536	4410

Este algoritmo de coloreado de grafos tiene una complejidad teórica lineal, teniendo en cuenta la restricción del enunciado donde cada nodo tiene a lo sumo 8 aristas, la cual se puede razonar de la siguiente forma:

N -> número de nodos del grafo

V -> vecinos de cada nodo

Recorremos con un bucle todos los nodos y a su vez recorremos todos los vecinos de dicho nodo por tanto tendríamos $O(N + V)$ pero como habíamos dicho antes, cada nodo tiene un máximo de 8 vecinos por tanto V en el peor caso es 8 (una constante) por tanto podríamos decir que la complejidad teórica del algoritmo es $O(n)$.

Los tiempos de la tabla corresponden con los esperados de la complejidad explicada anteriormente, teniendo un crecimiento lineal en función del número de nodos del grafo, pero teniendo una pequeña variación de tiempo en función del grado medio de todos los nodos.

Coloreado_grafo.py

```
n: 4 Tiempo(ms): 0.0
n: 8 Tiempo(ms): 1.0001659
n: 16 Tiempo(ms): 1.000404
n: 32 Tiempo(ms): 3.000259
n: 64 Tiempo(ms): 9.000301
n: 128 Tiempo(ms): 15.9983
n: 256 Tiempo(ms): 29.9997
n: 512 Tiempo(ms): 61.9997
n: 1024 Tiempo(ms): 121.99
n: 2048 Tiempo(ms): 248.00
n: 4096 Tiempo(ms): 522.00
n: 8192 Tiempo(ms): 1039.9
n: 16384 Tiempo(ms): 2207.
n: 32768 Tiempo(ms): 4727.
```

n / Tiempo	ColoreoGrafo.java
4	0
8	1
16	1
32	3
64	9
128	15
256	29
512	61
1024	121
2048	248
4096	522
8192	1039
16384	2207
32768	4727
65536	FdT

La implementación del algoritmo es prácticamente igual que la versión de java, lo único que haciendo una traducción de la sintaxis, por tanto el mismo razonamiento utilizado anteriormente se puede aplicar para deducir la complejidad de este, siendo también $O(n)$.

Los tiempos de la tabla también corresponden con los esperados a una complejidad lineal, aunque también teniendo en cuenta como habíamos explicado anteriormente la pequeña variación de tiempo en función del grado medio de todos los nodos.

----- PRACTICA 5 -----

PROGRAMACIÓN DINÁMICA

Ferry.java

1. Definición del estado: $dp[i][j]$

La tabla de datos tiene por filas i , el número de vehículos considerados, es decir los primeros i y por columnas la longitud total del barco ocupada en el carril de babor. La matriz es booleana por tanto que en un determinado i y j sea true, quiere decir que es posible colocar los primeros i vehículos ocupando una longitud j en babor.

La longitud ocupada de estribor no se necesita almacenar explícitamente ya que tenemos una variable que conoce el sumatorio de los primeros i vehículos, por tanto lo está longitud será igual al sumatorio de i menos la longitud total del barco.

2. Relación de recurrencia:

Caso base para calcular el estado de (i, j) a partir de los estados anteriores:

El vehículo cabe en babor si p (longitud de interacción) $\geq V_i$ y si además se cumple $dp[i-1][l-V_i]$ quiere decir que el vehículo guarda su estado en babor.

Por otro lado cabría en estribor si $S_i - l \geq L$ y si además se cumple $dp[i-1][l]$ entonces quiere decir que el vehículo guarda su estado en estribor.

3. Análisis de complejidad

La complejidad teórica del algoritmo en función de N (número de vehículos) y L (longitud del ferry) es $O(N * L)$, ya que tenemos que recorrer toda la matriz solución la cual tiene N filas y L columnas, y en cada una realizamos operaciones constantes.

----- PRACTICA 6 -----

Vuelta Atrás – Backtracking

AlmacenajeContenedores.java

Fichero	n elementos	tiempo (ms)
test00	7	180
test01	9	159
test02	15	180
test03	12	164
test04	20	199
test05	26	216
test06	14	170
test07	15	176
test08	41	327040
test09	66	FdT

Tiempo calculado con 100 repeticiones

¿Qué complejidad asigna al algoritmo que calcula el número mínimo de contenedores a utilizar? ¿Por qué?

Este algoritmo tiene una complejidad factorial $O(n!)$, ya que estamos realizando backtracking para explorar todas las posibles soluciones al problema, obtenemos esta complejidad debido a que para cada elemento que queremos añadir puede meterse en cualquiera de los contenedores existentes o crear uno nuevo, y así con todos los elementos, por tanto a medida que avanzamos añadiendo elementos, el número de contenedores posibles crece, de tal forma que el número de posibles configuraciones al añadir un elemento más, serán las anteriores multiplicado por las nuevas posibles que surgen con el nuevo elemento, lo cual se puede traducir a que sigue un crecimiento factorial.

----- **PRACTICA 6 Extra** -----

Problema NReinas

NReinasTiempos.java

Tablero NxN	tiempo (ms)	n soluciones
4	6	2
5	0	10
6	0	4
7	0	40
8	1	92
9	3	352
10	11	724
11	61	2680
12	317	14200
13	1832	73712
14	11145	365596
15	73228	2279184

```
C:\Users\U03988387\Desktop\alg_CienFuegosAlvarezAlb
Se encontraron 2 soluciones para N = 4
Tablero de tamaño 4 --- Tiempo(ms): 6

Se encontraron 10 soluciones para N = 5
Tablero de tamaño 5 --- Tiempo(ms): 0

Se encontraron 4 soluciones para N = 6
Tablero de tamaño 6 --- Tiempo(ms): 0

Se encontraron 40 soluciones para N = 7
Tablero de tamaño 7 --- Tiempo(ms): 0

Se encontraron 92 soluciones para N = 8
Tablero de tamaño 8 --- Tiempo(ms): 1

Se encontraron 352 soluciones para N = 9
Tablero de tamaño 9 --- Tiempo(ms): 3

Se encontraron 724 soluciones para N = 10
Tablero de tamaño 10 --- Tiempo(ms): 11

Se encontraron 2680 soluciones para N = 11
Tablero de tamaño 11 --- Tiempo(ms): 61

Se encontraron 14200 soluciones para N = 12
Tablero de tamaño 12 --- Tiempo(ms): 317

Se encontraron 73712 soluciones para N = 13
Tablero de tamaño 13 --- Tiempo(ms): 1832

Se encontraron 365596 soluciones para N = 14
Tablero de tamaño 14 --- Tiempo(ms): 11145

Se encontraron 2279184 soluciones para N = 15
Tablero de tamaño 15 --- Tiempo(ms): 73228
```

Complejidad del algoritmo:

El algoritmo tiene una complejidad factorial $O(n!)$ en función del tamaño del tablero donde se colocan las reinas, utilizando la técnica de backtracking. Esto se obtiene de que vamos colocando una reina por fila y probando las posibles columnas disponibles en cada paso, por lo que el número de configuraciones se va multiplicando por cada posible solución, lo que da lugar a un crecimiento factorial.

Observación:

A partir de $N = 16$ el programa produce una excepción debido a que se llena el espacio de memoria del heap debido a la gran cantidad de soluciones que debe ir almacenando y las numerosas llamadas recursivas que debe ir gestionando.

```
Se encontraron 365596 soluciones para N = 14  
Tablero de tamaño 14 --- Tiempo(ms): 11145
```



```
Se encontraron 2279184 soluciones para N = 15  
Tablero de tamaño 15 --- Tiempo(ms): 73228
```



```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space  
    at java.base/java.util.Arrays.copyOfOf(Arrays.java:3509)  
    at java.base/java.util.Arrays.copyOfOf(Arrays.java:3478)  
    at java.base/java.util.ArrayList.grow(ArrayList.java:238)  
    at java.base/java.util.ArrayList.grow(ArrayList.java:245)  
    at java.base/java.util.ArrayList.add(ArrayList.java:484)  
    at java.base/java.util.ArrayList.add(ArrayList.java:497)  
    at NReinas.backtracking(NReinas.java:67)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.backtracking(NReinas.java:87)  
    at NReinas.resolverNReinas(NReinas.java:35)  
    at NReinasTiempos.probarTest(NReinasTiempos.java:24)  
    at NReinasTiempos.main(NReinasTiempos.java:13)
```

----- PRACTICA 7 -----

Ramifica y Poda – Branch and Bound

Heurístico:

Primero hemos aplicado como heurístico de construcción, la ordenación de los elementos a insertar de forma que primero colocamos los elementos más grandes, que son los que nos van a restringir el espacio disponible en los contenedores y así detectar antes configuraciones inviables.

Luego el heurístico de ramificación usado para la generación del árbol de estados del problema es primero tratar de meter los elementos en contenedores existentes haciendo que se generen primero los nodos donde no aumentamos k (el número de contenedores), al finalizar trataremos de crear un nuevo contenedor en caso de poder mejorar la solución actual, dejando esto como última opción, todo esto nos permite encontrar buenas soluciones iniciales y mejorar la eficiencia del heurístico de poda que hemos utilizado.

Este heurístico de poda consiste en calcular una cota inferior del número mínimo teórico de contenedores adicionales necesarios para completar el problema en base a la suma de los elementos que quedan por introducir. Si la suma del número de contenedores actuales más esta cota es mayor o igual que la mejor solución hasta ahora, podamos la rama, ya que no conseguiremos mejorarla.

Estos dos heurísticos en conjunto permiten reducir en gran medida el árbol de búsqueda ya que corta de raíz las ramas que incluso en el mejor de los casos no pueden mejorar la solución actual.

AlmacenajeContenedoresRyP.java

Fichero	n elementos	tiempo (ms) SIN RyP	tiempo (ms) CON RyP
test00	7	180	157
test01	9	159	149
test02	15	180	163
test03	12	164	146
test04	20	199	167
test05	26	216	211
test06	14	170	165
test07	15	176	170
test08	41	327040	263
test09	66	FdT	364

Tiempo calculado con 100 repeticiones



Como podemos observar en la comparación de tiempos conseguimos rebajar sensiblemente el tiempo medio de ejecución para obtener la solución en todos los casos, utilizando estos heurísticos de ramificación y poda, incluso conseguimos obtener la solución del caso 8 (que antes tardaba muchísimo) y del caso 9 (que antes se marcó como FdT) en un tiempo razonable ligeramente superior al resto de casos, gracias a la poda de raíz del árbol de búsqueda.

FIN.

Alberto Cienfuegos Álvarez